

APC Assignment 1 – October 2025

1 Introduction

Ordinary Differential Equations (ODEs) are mathematical models describing the evolution of a system whose rate of change depends on its current state. Formally, let $I = [t_0, t_F] \subset \mathbb{R}$, and consider a continuously differentiable function $f : I \times \mathbb{R}^n \rightarrow \mathbb{R}^n$ and an initial value $\mathbf{y}_0$. The *Initial Value Problem* (IVP) is then defined as

$$\begin{cases} \mathbf{y}'(t) = f(t, \mathbf{y}(t)), & t \in I, \\ \mathbf{y}(t_0) = \mathbf{y}_0, \end{cases} \quad (1)$$

where $\mathbf{y}(t)$ is the unknown vector-valued function that evolves over time t . Under the assumption that f satisfies a Lipschitz continuity condition with respect to $\mathbf{y}$, existence and uniqueness of the solution $\mathbf{y}(t)$ are guaranteed.

1.1 Runge–Kutta Methods

Among the many techniques used to approximate the solution of IVPs, the *Runge–Kutta* (RK) methods form a widely used family of *single-step* numerical schemes. In these methods, each new approximation $\mathbf{u}_{n+1}$ depends directly only on the previous value $\mathbf{u}_n$, and is computed as a weighted average of several intermediate evaluations of f . Specifically,

$$\mathbf{u}_{n+1} = \mathbf{u}_n + h \sum_{i=0}^{s-1} b_i \mathbf{k}_i, \quad (2a)$$

$$\mathbf{k}_i = f \left(t_n + c_i h, \mathbf{u}_n + h \sum_{j=0}^{s-1} a_{ij} \mathbf{k}_j \right), \quad (2b)$$

where s denotes the number of stages, h is the integration step, and the coefficients a_{ij} , b_i , and c_i define the specific RK variant. These coefficients are commonly represented in the so-called *Butcher tableau*:

c_0	$a_{0,0}$	$\dots$	$a_{0,s-1}$	with $c_i = \sum_j a_{ij}$.
$\vdots$		$\ddots$		
c_{s-1}	$a_{s-1,0}$	$\dots$	$a_{s-1,s-1}$	
	b_0	$\dots$	b_{s-1}	

In the following, for simplicity, we assume that the coefficients a_{ij} always form a lower triangular matrix, i.e.,

$$a_{ij} = 0 \quad \text{for } j > i, \quad i, j = 0, \dots, s-1$$

RK methods can be divided into two main classes: explicit and implicit schemes. The distinction arises from the entries of the Butcher tableau. For $i = 0, \dots, s-1$, stage i of an RK method is *explicit* if the corresponding diagonal coefficient is also zero, that is, $a_{ii} = 0$ holds. In this case, $\mathbf{k}_i$ depends only on the previously computed stages $\mathbf{k}_j$ with $j < i$, and the result of Equation (2b) can be computed directly in closed form. By contrast, an *implicit* RK stage has $a_{ii} \neq 0$, so that the stage depends on itself. Computing $\mathbf{k}_i$ for such a stage generally requires solving a nonlinear system, often via an iterative *fixed-point algorithm*. The single-step algorithm, which accounts for both explicit and implicit stages, is described in the following pseudo-code:

Algorithm 1 Single Runge–Kutta Step

Input: current time t , current solution $\mathbf{u}_n$, current step h

Available: Butcher tableau coefficients $\{a_{ij}\}_{ij}$, $\{b_i\}_i$, $\{c_i\}_i$, number of stages s , IVP function f

Output: single-step solution $\mathbf{u}_{n+1}$

```

1: for  $i = 0$  to  $s - 1$  do
2:   compute  $\mathbf{S}_i = \sum_{j < i} a_{ij} \mathbf{k}_j$ 
3:   if stage  $i$  is implicit then                                     ▷ implicit stage
4:     compute  $\mathbf{k}_i$  via a fixed-point iterative algorithm
5:      $\mathbf{S}_i \leftarrow \mathbf{S}_i + a_{ii} \mathbf{k}_i$ 
6:   else                                                             ▷ explicit stage
7:      $\mathbf{k}_i \leftarrow f(t + c_i h, \mathbf{u}_n + h \mathbf{S}_i)$                      ▷ Equation (2b)
8:   end if
9: end for
10: compute  $\mathbf{u}_{n+1}$  via Equation (2a)
11: return  $\mathbf{u}_{n+1}$ 

```

1.2 Adaptive Runge–Kutta

Fixed-step methods can be inefficient when the solution exhibits regions of varying smoothness. *Adaptive Runge–Kutta methods* address this issue by dynamically adjusting the step size h_n at each iteration n , based on an estimate of the local error. Typically, smaller steps are used near steep gradients and larger ones where the solution varies slowly.

An *a posteriori* estimate of the local error can be obtained by comparing two candidate approximations: one ($\mathbf{u}_{h_n}$) computed with a single step of size h_n , and another ($\mathbf{u}_{h_n/2}^{(2)}$) obtained through two steps of size $h_n/2$. Given the current approximation $\mathbf{u}_n$, if the relative error E_n (measured in infinity norm)

satisfies the following condition

$$E_n = \frac{\|\mathbf{u}_{h_n/2}^{(2)} - \mathbf{u}_{h_n}\|_\infty}{\|\mathbf{u}_n\|_\infty} < \frac{\varepsilon}{2}, \quad (3)$$

for a prescribed tolerance ε , the step is accepted and $\mathbf{u}_{h_n/2}^{(2)}$ is taken as the new approximation. Otherwise, the step size is halved, but it never decreases below $h_{\min}$, a user-defined lower bound. Also, to accelerate the computation, the step size is doubled whenever the error falls below $\varepsilon/2^{s+1}$. The complete adaptive algorithm is displayed in the following pseudo-code:

Algorithm 2 Adaptive Runge–Kutta integration

Available: initial condition $\mathbf{u}_0$, interval $[t_0, t_F]$, initial step h , minimum step $h_{\min}$, error tolerance ε , number of stages s

Output: discrete solution $\{\mathbf{u}_n\}$ and times $\{t_n\}$

```

1:  $n \leftarrow 0, t \leftarrow t_0, h_n \leftarrow h$ 
2: while  $t_n + h_n \leq t_F$  do
3:   compute  $\mathbf{u}_{h_n}$  from  $\mathbf{u}_n$  via one Algorithm 1 step with size  $h_n$ 
4:   compute  $\mathbf{u}_{h_n/2}^{(2)}$  from  $\mathbf{u}_n$  via two Algorithm 1 steps with size  $h_n/2$ 
5:   compute  $E_n$  via Equation (3)
6:   if  $E_n < \varepsilon/2$  or  $h_n < h_{\min}$  then
7:      $t_{n+1} \leftarrow t_n + h_n$ 
8:      $\mathbf{u}_{n+1} \leftarrow \mathbf{u}_{h_n/2}^{(2)}$ 
9:     if  $E_n < \varepsilon/2^{s+1}$  then
10:       $h_n \leftarrow 2h_n$ 
11:    end if
12:     $n \leftarrow n + 1$ 
13:   else ▷ step rejected
14:      $h_n \leftarrow h_n/2$ 
15:   end if
16: end while
17:  $h_n \leftarrow t_F - t_n$ 
18: if  $h_n > (t_F - t_0)/10^3$  then ▷  $h_n$  is non-zero (with numerical tolerance)
19:    $t_{n+1} \leftarrow t_F$ 
20:   compute  $\mathbf{u}_{n+1}$  via two Algorithm 1 steps with size  $h_n/2$ 
21: end if
```

Note that after the adaptive loop, we perform one last step to complete the integration in the rightmost portion of the interval, provided that the width of the remaining portion is strictly larger than zero.

2 Code implementation

In the file `RKSolver.cpp`, you will find a skeleton of the code to be completed. After carefully reading the provided files and the documentation within and

understanding the class structure, you must implement the following two class methods, marked by the comment *Your code goes here* in `RKSolver.cpp`:

```
Rnvector RKSolver::single_step(const double tn, const Rnvector &un,
    const double hn) const;
```

```
void RKSolver::solve();
```

These functions correspond to the procedures described in Algorithms 1 and 2, respectively. The first method *returns* the single-step solution $\mathbf{u}_{n+1}$, while the second computes the discrete solution $\{\mathbf{u}_n\}$ and times $\{t_n\}$ by filling the appropriate class members.

The `utils.hpp` file defines several useful aliases and operators to be used in the implementation, such as the `Rnvector` alias (the return type of the `single_step` method). Furthermore, the `RKSolver` class (see, in particular, `RKSolver.hpp`) contains as members the quantities available in the above pseudo-code that are not function arguments of the requested methods, such as the number of stages, as well as a fixed-point algorithm for implicit stages. Finally, the classes defined in `equations.hpp` and `BaseEquation.hpp` represent the differentiable function f and the entire IVP of Equation (1), respectively.

Remark: the `<cmath>` library (already included in the project) provides the following functions, which you can use in your code to compute the power of 2 and the absolute value of a number, respectively:

```
double exp2 ( double num );
double std::abs ( double x );
```

In addition, the `<algorithm>` library provides the following function:

```
ForwardIt std::max_element ( ForwardIt first, ForwardIt last );
```

This function takes as arguments two iterators (pointers) to the first and last elements of an `std::vector` (or any generic container) and returns an iterator to the largest element in the range.

All other methods defined in the code are also available for use.

Important: only the `RKSolver.cpp` file should be modified. Your implementations must be consistent with the method declarations and the class structure of the rest of the project. As mentioned, you may rely on the functions that are already implemented. The `main.cpp` file allows you to run several integration algorithms on a test function (the f in Equation (1)). In particular, the `run_test` function includes two sets of pre-loaded Butcher tableaus: one corresponding to the `Heun` method and one to `RK4` (4-stage Runge–Kutta). Executing the `main` will write the results for two example IVPs to the `solution_1_Heun.txt` and `solution_1_RK4.txt` files, copies of which are included in the zipped folder for comparison with the assignment solution. These files have the following structure:

Solution:

```
0
-0.00944853
-0.0190458
[...]
-77.1711
-87.5194
Time instants:
0 0.003125 0.00625 [...] 0.975 1
```

Important: during the evaluation, we will test your code on the provided test function *and* on additional ones that are not available to you.

3 Delivery instructions

This assignment is not mandatory. However, if your implementation is correct, it will earn you an additional +1 point on the final grade.

Please follow these instructions for submission:

- Download the zipped folder `Assignment1_initial.zip` from WeBeep.
- Unzip the folder.
- Rename the **unzipped** folder as `YourCodicePersona` (e.g., `10699999`).
Important: do this **before** re-zipping the project with your solution.
- Implement your code in the provided files.
- Test the code by executing the `main` and comparing the results from your files with the provided baseline.
- Zip the entire project folder again, making sure that the resulting file name is `YourCodicePersona.zip` (e.g., `10699999.zip`).
- Upload the file on WeBeep → Assignments → Assignment 1.

Attention: This assignment is individual; no group or team work is allowed. In cases of plagiarism, a penalty of 2 points (i.e., -2 on the total score) will be applied, and you will be excluded from future assignments. You are fully responsible for the code you submit. Using generative AI tools may produce code that closely resembles that of other students, potentially triggering plagiarism detection and leading to the penalties described above.

If you have questions, please post them on the WeBeep Assignments forum. **No feedback will be provided in the last 24 hours before the deadline.**

Code submission opens on Thursday, October 16th at 18:00 and closes on Wednesday, October 22nd at 23:59 (Rome time).

If any updates are made to the provided source code, you will be informed through an announcement. Make sure to regularly check WeBeep during the submission period.